

Assignment 4: Defense against CI Intrusions

CSCI 5743, Fall 2025

Total Points: 200

Learning Objectives

By completing this assignment, students will be able to:

- **Identify and evaluate** security failures in real-world cyber incidents using core cyber defense principles such as Least Privilege, Separation of Duties, Fail-Safe Defaults, and Network Segmentation.
- **Map** attacker techniques from the MITRE ATT&CK framework to appropriate countermeasures in the MITRE D3FEND framework to demonstrate layered defense thinking.
- **Relate** high-level cybersecurity outcomes from the NIST Cybersecurity Framework (CSF 2.0) to operational security controls from NIST SP 800-53 Rev. 5, within the context of critical infrastructure defense.
- **Apply** host-based firewall configurations using `iptables` to enforce network control policies, minimize exposure, and analyze their effect on system communication and accessibility.
- **Explain** the role and impact of stateful packet inspection in managing and securing connections in TCP/IP-based communications.
- **Demonstrate** practical skills in building and evaluating a simple decision tree classifier using Python, pandas, and scikit-learn to detect anomalous login behaviors.
- **Compare and interpret** key classification performance metrics, accuracy, precision, recall, and F1-score, and justify their use in security-sensitive detection systems.
- **Reflect critically** on the trade-offs between false positives and false negatives in real-world security applications, and assess model behavior based on feature importance and classifier depth.
- **Interpret machine learning results** in the context of cybersecurity decision-making, particularly when using anomaly detection for login activity analysis.
- **Implement** a secure file exchange system using hybrid cryptography (AES-CTR for confidentiality, HMAC for integrity, and RSA-OAEP for key protection).

- **Explain** cryptographic design decisions such as encrypt-then-MAC, IV reuse risks, and RSA-OAEP padding schemes in the context of secure data transmission.
- **Connect** cryptographic system design to relevant NIST CSF functions and SP 800-53 Rev. 5 controls to demonstrate policy-to-implementation alignment.

Submission Guidelines

1. Report

- Download the provided [Assignment 4 report file](#).
- Complete the report by filling out all required sections, ensuring all **tasks and deliverables** are documented clearly.
- **Submit both** the completed `a4_report.md` and the converted `a4_report.pdf`.
- **The PDF will be graded; the Markdown file will be retained for reference.**

2. Screenshots

- Include **all required screenshots** referenced in the report.
- Place all screenshots in a subfolder named `screenshots` within your submission folder.
- **Embed screenshots** directly in the Markdown report using the syntax below:

```
![Screenshot Description](./screenshots/filename.png)
```

- Ensure screenshots are **clear, legible, and uncropped**.

3. Supporting Files (Required)

You must include **all supporting code and output files** required to verify your work.

To maintain organization, separate these materials by task within the `extras/` folder.

The directory should follow this structure:

Assignment4_lastName.zip

```
├─ a4_report.md
├─ a4_report.pdf
├─ screenshots/
│   ├── task1_firewall_rules.png
│   ├── task2_classifier_output.png
│   ├── task3_crypto_success.png
│   └─ ...
├─ extras/
│   ├── task1_firewall/
│   │   ├── iptables_rules.txt
│   │   └─ any_relevant_logs.txt
│   └─
├─ task2_classifier/
│   └─ classifier.py
├─
├─ task3_crypto/
│   ├── keygen.py
│   ├── encrypt.py
│   ├── decrypt.py
│   ├── ciphertext.bin
│   ├── tag.bin
│   ├── wrapped_keys.bin
│   └─ decrypted.txt
└─ README.txt (optional - briefly describe contents of each subfolder)
```

Note:

- All scripts must execute correctly in a Python 3 environment.
- Include any configuration files, logs, or outputs referenced in your report.

4. Submission Format

- Submit your completed assignment as a single compressed **Assignment4_lastName.zip** file with the above structure.
- Ensure all paths in your Markdown report correctly reference files inside this structure.
- Converting the `.md` report to **PDF is required**.
- Ensure all formatting, code blocks, tables, and screenshots are preserved in the PDF.
- The PDF version will be used for grading.

5. Final Submission

- Upload `Assignment4_lastName.zip` (containing the report, screenshots, and all supporting files) to the course submission portal by the specified deadline.
- Late submissions are subject to course policy on penalties.



Grade Breakdown

Part	Task	Points	Description
Part 1: Conceptual Questions	Task 1: Real-World Breach Analyses	30 pts	Analyze 5 major breaches using cyber defense principles.
	Task 2: MITRE ATT&CK to D3FEND Mapping	20 pts	Map 5 ATT&CK techniques to D3FEND countermeasures.
	Task 3: NIST CSF to SP 800-53 Mapping	15 pts	Map CSF functions to SP 800-53 Rev. 5 controls for CI security implementation.
Part 2: Hands-On Lab	Task 1: Firewall Configuration and Analysis	45 pts	Apply and analyze iptables firewall rules using stateful filtering.
	Task 2: Login Anomaly Detection with ML	45 pts	Build and evaluate a decision tree classifier to detect anomalous login events.
	Task 3: Secure File Exchange (Hybrid Crypto)	45 pts	Implement AES-CTR + HMAC + RSA-OAEP for encrypted and authenticated file transfer.
Total		200 pts	

Part 1: Conceptual Questions

Task 1: Analyze Violation of Cyber Defense Principles in Key Real-world Breaches (30 pts).

Objective

In this task, you will examine high-profile real-world cyber breaches and analyze how fundamental cybersecurity defense principles were violated. By identifying these failures, you will evaluate how applying principles such as Least Privilege, Separation of Duties, Fail-Safe Defaults, and Continuous Monitoring could have reduced risk, limited attacker movement, or prevented the breach entirely.

1. Capital One Cloud Breach (2019)

Case Summary: This breach occurred when a misconfigured AWS Web Application Firewall (WAF) role was exploited using a Server-Side Request Forgery (SSRF) vulnerability. The attacker was able to use the WAF's permissions to access and exfiltrate sensitive data from Amazon S3 buckets. The role had far broader access than necessary and combined multiple responsibilities that should have been kept separate. Additionally, the activity went unnoticed for an extended period, suggesting a lack of effective detection mechanisms or alerting on abnormal access behavior. Tighter control over assigned permissions, clearer separation of operational responsibilities, and more robust monitoring could have limited the attacker's movement or exposed the breach much earlier.

Prompt: Explain how principles like *Least Privilege*, *Separation of Duties*, and *Continuous Monitoring* were violated in this breach, and how would proper enforcement of these principles have limited or prevented unauthorized data access?

2. Equifax Breach (2017)

Case Summary: Attackers exploited a known vulnerability in Apache Struts that had remained unpatched on a public-facing web application. This allowed them to gain remote access and move laterally within the network, ultimately stealing personal information of 147 million individuals. The affected system was exposed to the internet and included components that were not essential to its core function, increasing its vulnerability. Once inside, the attackers encountered few restrictions or internal safeguards. If the system had been configured to block external access by default and had

only the necessary services enabled, the attack vector would have been far more limited or completely inaccessible.

Prompt: Analyze the role of *Fail-Safe Defaults* and *Least Functionality* in this breach, and how would applying these principles have prevented external access to the vulnerable application?

3. Target Breach (2013)

Case Summary: The attackers gained initial access to Target's network using credentials stolen from a third-party HVAC vendor. Once inside, they moved laterally through a flat internal network and ultimately reached the point-of-sale systems to steal customer payment data. The lack of internal boundaries and unrestricted trust between systems allowed the attackers to traverse the network without encountering significant barriers. There was no clear separation between third-party systems and critical business infrastructure, and minimal checks on internal communications. Introducing internal boundaries, limiting trust relationships, and isolating critical systems from external access points could have significantly reduced the attack's reach.

Prompt: Identify the failures in *Network Segmentation*, and *Zero Trust*. How would implementing stronger segmentation and stricter access verification have blocked lateral attacker movement?

4. SolarWinds Supply Chain Attack (2020)

Case Summary: Attackers compromised the SolarWinds Orion software build system and inserted a backdoor (SUNBURST) into updates deployed by thousands of customers. This allowed stealthy lateral movement and long-term persistence across high-value targets, including government agencies. Lack of software supply chain controls and limited internal monitoring contributed to the undetected spread of the compromised software.

Prompt: Discuss the lack of *Zero Trust*, *Continuous Monitoring*, and *Open Design*. How could implementing these principles have contained the spread or improved detection?

5. Colonial Pipeline Ransomware (2021)

Case Summary:

The attacker gained entry to Colonial Pipeline's internal systems using a valid VPN account that did

not require multi-factor authentication. Once inside, the lack of authentication depth and absence of real-time monitoring allowed the intruder to remain undetected while deploying ransomware that disrupted fuel distribution across the Eastern United States. User activity was not adequately logged or attributed, and there were insufficient safeguards to verify identity or restrict movement post-login. If access had required additional verification steps and if behaviors were actively monitored and flagged, the intrusion could have been detected or blocked earlier. Robust access controls and layered protections would have made it more difficult for a single compromised account to lead to such a high-impact event.

Prompt: Explain the breakdown of *Accountability and Non-repudiation*, *Continuous Monitoring*, and *Layered Defense*. How would enforcing these principles-especially MFA and audit logging-have changed the attacker's access path?

Task 2: MITRE ATT&CK to D3FEND Mapping (20 pts)

Objective

The MITRE D3FEND framework (<https://d3fend.mitre.org/>) is a structured knowledge base of cybersecurity countermeasures that directly complements the MITRE ATT&CK framework. While ATT&CK catalogs the techniques adversaries use during various stages of a cyber intrusion, D3FEND provides a taxonomy of defensive techniques that organizations can implement to detect, prevent, deceive, isolate, or recover from those attacks.

D3FEND is organized into six major categories of defensive action:

- Harden – Strengthen systems and configurations to reduce attack surfaces
- Detect – Identify signs of malicious activity through analysis and monitoring
- Isolate – Segment or restrict access to limit attacker movement and exposure
- Deceive – Mislead attackers using fake data, resources, or environments
- Evict – Remove malicious artifacts, accounts, or sessions from the environment
- Restore – Recover access or functionality after an attack

This assignment asks you to explore the relationships between these two frameworks by mapping specific ATT&CK techniques used in a simulated attack scenario to relevant D3FEND countermeasures. Your goal is to analyze how layered defensive strategies can be applied to mitigate real-world adversarial behavior.

Read the following scenario of a hypothetical APT intrusion:

An attacker is preparing to compromise an enterprise network. They begin by conducting open-source intelligence (OSINT) activities-searching social media, corporate websites, and public data sources to collect information about employees, technologies in use, and possible exposed services. Using this intelligence, they craft a spearphishing email with a malicious attachment tailored to a specific employee.

Once the employee opens the attachment, malware is executed to establish initial access. The attacker then installs a backdoor to ensure persistence, even if the initial vector is discovered and removed. To evade detection by endpoint protection systems and security monitoring tools, the attacker uses obfuscation techniques such as packing and encoding.

The attacker then performs internal reconnaissance to discover additional systems, credentials, and privileged access opportunities within the network. After identifying valuable data, they exfiltrate it over an encrypted command-and-control (C2) channel, making it difficult to detect via traditional traffic inspection.

Your Task:

1. Identify Five MITRE ATT&CK Techniques

Based on the above scenario, identify **five attack techniques** used in this intrusion, each belonging to a different **MITRE ATT&CK tactic**. For each technique:

- Provide the **tactic name** (e.g., *Initial Access*, *Persistence*, etc.)
- Provide the **technique name and technique ID** (e.g., *Spearphishing Attachment (T1566.001)*)
- Include a **brief explanation (1–2 sentences)** that describes how the technique is used in the given scenario

2. Map Each ATT&CK Technique to Two D3FEND Countermeasures

For each of the five selected ATT&CK techniques, identify **two distinct D3FEND techniques** from any of the following defensive categories:

- Harden
- Detect
- Isolate
- Deceive
- Evict
- Restore

For each D3FEND technique:

- Provide the **technique name**
- Indicate the **category** it belongs to

- Write a **brief justification** explaining how or why this technique would mitigate, detect, or prevent the specific attack technique selected

Example:

ATT&CK Tactic: Initial Access

Technique: Spearphishing Attachment (T1566.001)

Explanation: The attacker sends a crafted phishing email with a malicious file attachment to an employee in order to execute malware and gain an initial foothold.

D3FEND Countermeasures:

1. **Email Filtering** – *Isolate*

- Prevents delivery of emails containing malicious content through rule-based or ML-driven filtering engines.

2. **Message Authentication** – *Harden*

- Ensures email integrity and authenticity using mechanisms like SPF, DKIM, or DMARC, making it more difficult for attackers to spoof legitimate domains.

Task 3: Applying the NIST Cybersecurity Framework (CSF) and SP 800-53 Rev. 5 Controls (15 pts)

Objective

In this task, you will explore how **high-level cybersecurity outcomes** from the **NIST Cybersecurity Framework (CSF 2.0)** map to **specific security controls** from **NIST SP 800-53 Rev. 5**.

This exercise links organizational strategy (CSF) to operational implementation (SP 800-53).

Scenario

A regional **critical-infrastructure operator** (e.g., water utility or energy provider) is modernizing its operational network after several intrusion attempts.

The organization wants to use the **NIST CSF Functions** to shape its security strategy and then choose concrete **SP 800-53 controls** to implement those outcomes.

Your Task

1. **Select any 3 CSF Functions** from the six:
- Govern

• Identify

• Protect

• Detect

• Respond

• Recover
2. For **each selected Function**, choose **one Category or Subcategory** from the CSF 2.0 Core (see: [NIST CSF 2.0 Core](#)).
3. For each chosen Category/Subcategory:
- Identify **one or two SP 800-53 Rev. 5 controls** that directly support it.
(Use [csf.tools SP 800-53 catalog](#).)

• Briefly explain **how that control operationalizes the CSF outcome** in a critical-infrastructure context.

Example Response

CSF Function	Category / Subcategory	Related SP 800-53 Controls	Explanation
Protect	PR.AA-02 - Access is limited to authorized users, processes, and devices.	AC-2 - Account Management , IA-2 - Identification and Authentication (Org Users)	AC-2 and IA-2 enforce identity lifecycle and MFA, ensuring only approved accounts access OT systems.
...	...	...	...
...	...	...	...

Part 2: Hands-on - Defending against Cyber Intrusions

Task 1: Host-Based Firewall Setup and Testing Using iptables (45 pts)

Objective:

In this task, you will configure host-based firewall rules using `iptables` on the **Defense VM** to enforce minimal exposure and controlled service access. You will then analyze and justify the changes in system behavior resulting from these firewall rules.

Testbed Configuration

In this part, you will use our developed testbed in prior assignments. We need a two-VM setup to simulate a simple DMZ environment and an attacker. Make sure your virtual network is properly isolated and both VMs are configured to communicate with each other.

Required VMs

- **Attack VM:** Attack Kali Linux
- **Defense VM:** Defense Kali Linux

Network Setup

- Ensure that both VMs are connected to the **same isolated internal network**.

Verification Steps

1. Confirm that both VMs can **ping each other**.
2. Use `ip a` on both machines to verify IP addresses and ensure they're in the same subnet (e.g., `192.168.10.x`).
3. Record the IP addresses of both the Attack and Defense VMs, as you will need them when writing specific firewall rules and during testing.

Step 1: Baseline Behavior (Before Firewall Setup)

Before applying any firewall rules, verify that all traffic is permitted and services are reachable under default conditions. This establishes a baseline for comparing changes after the firewall is applied.

1. ICMP Flood Test (Ping Flood)

- From the **Attack VM**, run a ping flood to the **Defense VM**:

```
ping -f <defense-vm-ip>
```

 If `ping -f` requires elevated privileges, use `sudo`.

- On the **Defense VM**, launch **Wireshark**, start capturing on the network interface (e.g., `eth0`), and filter using:

```
icmp
```

- Confirm that **ICMP echo-request packets** are received continuously.
- Take a screenshot showing the ICMP flood activity in Wireshark.

2. Web Server Availability

- On the **Defense VM**, start a basic HTTP server:

```
sudo python3 -m http.server 80
```

- On the **Attack VM**, confirm HTTP access:

```
curl http://<defense-vm-ip>
```

- Or verify via `nmap`:

```
nmap -p 80 <defense-vm-ip>
```

3. SSH Service Availability

- Ensure the SSH service is running on the **Defense VM**:

```
sudo systemctl start ssh
```

```
sudo systemctl status ssh
```

- From the **Attack VM**, scan for port 22:

```
nmap -p 22 <defense-vm-ip>
```

- Optionally, attempt to SSH into the Defense VM (if credentials are known and permitted).

Step 2: Apply Firewall Rules on Defense VM

In this step, you will configure a host-based firewall using `iptables` on the **Defense VM** to restrict traffic to essential services only, enhancing its security posture in an isolated network.

Firewall Rule Requirements

1. Allow Incoming HTTP (Port 80) from Any Source

- Accept new and established TCP connections to port 80.

2. Allow Outgoing ICMP Echo-Request and Incoming Echo-Reply

- Permit only outgoing ICMP echo-request packets.
- Allow only incoming ICMP echo-reply packets.
- All other ICMP types must be blocked.

3. Block All Other Traffic

- Any traffic not explicitly allowed must be dropped due to the default DROP policy (applied earlier or as part of this task).

Commands to Apply Rules

```
# Allow incoming HTTP (port 80) from any source
sudo iptables -A INPUT -p tcp --dport 80 -m state --state NEW,ESTABLISHED -j ACCEPT
sudo iptables -A OUTPUT -p tcp --sport 80 -m state --state ESTABLISHED -j ACCEPT

# Allow outgoing ICMP echo-requests (ping) and allow replies
sudo iptables -A OUTPUT -p icmp --icmp-type echo-request -j ACCEPT
sudo iptables -A INPUT -p icmp --icmp-type echo-reply -j ACCEPT
```

Add Default Drop Policies

```
# Default policies (if not already applied)
sudo iptables -P INPUT DROP
sudo iptables -P OUTPUT DROP
```

Verification and Output

- Display the configured rules:

```
sudo iptables -L -v -n
```

- Take a screenshot of the rule list and save it for your submission.

Step 3: Post-Firewall Behavior and Analysis

After applying the firewall rules from step 2, re-run the tests from step 1 and evaluate how the system behavior has changed. Document your observations with evidence and reasoning.

1. Repeat the Ping Flood Test

- From the **Attack VM**, repeat the ICMP flood to the **Defense VM**:

```
ping -f <defense-vm-ip>
```

- On the **Defense VM**, use **Wireshark** with the filter:

```
icmp
```

- Observe whether **echo-requests** are still being received or blocked.
- Take a screenshot of the capture and describe what ICMP types (if any) are visible.

2. Repeat Service Availability Scan

- From the **Attack VM**, run:

```
nmap -p 80,22 <defense-vm-ip>
```

- Confirm and document:
 - Whether **HTTP (port 80)** is still accessible.
 - Whether **SSH (port 22)** is now blocked as expected.
- Include screenshots or logs from the scan output.

3. ICMP Directionality Test

- From the **Attack VM**, run:

```
ping <defense-vm-ip>
```

- From the **Defense VM**, run:

```
ping <attack-vm-ip>
```

- Record:
 - Which ping attempts succeed.
 - Which are dropped or time out.
- Explain these outcomes in terms of the firewall rules.

Analysis Questions

Write a well-reasoned response addressing the following questions. Support your answers with observations from your tests and packet captures.

Stateful Filtering

1. Why do we specify `NEW, ESTABLISHED` in the **INPUT** chain for destination port 80?

- Explain the role of stateful inspection in allowing both new inbound HTTP requests and the continuation of existing TCP sessions.
- Why would omitting `ESTABLISHED` affect the behavior of legitimate connections?

2. Why does the rule on the **OUTPUT** chain only include `ESTABLISHED` but not `NEW` ?

- Describe why initiating outbound connections for port 80 is unnecessary in this context, and how state tracking ensures response packets are allowed out.

3. What would be the security implications if state tracking (`-m state`) was not used in this setup?

- Would omitting stateful inspection open the system to spoofed ACK or SYN-ACK packets, TCP session hijacking, or other attacks?

4. Why do we allow only echo-request (outbound) and echo-reply (inbound) for ICMP, without using stateful filtering?

- Explain why ICMP is not tracked with state modules, and why explicitly filtering by type is preferred in this scenario.

Firewall Rules Impact

5. What changes did you observe in service accessibility after applying the firewall rules?

- Compare service availability before and after firewall enforcement.
- Include specifics such as ping directionality, SSH being blocked, and continued HTTP accessibility.

6. How do these observations demonstrate the effectiveness of the configured firewall?

- Did the rules perform as intended?
- Were there any unexpected behaviors, missed blocks, or incorrectly allowed traffic?

7. Based on your findings, what additional rules might be recommended for a production web server in the DMZ?

- Consider whether DNS, HTTPS, or logging/rate-limiting rules would be necessary for a real-world deployment.

Cyber Defense Principles

8. In what ways do these firewall rules enforce the principle of *attack surface minimization*?

- Describe how restricting access to only essential services reduces the visibility of the server to potential attackers.

9. How does the configuration embody the principle of *least privilege*?

- Explain how the firewall grants only necessary access and avoids overly permissive rules.

10. How does this configuration reflect the idea of *fail-safe defaults*?

- Discuss the importance of default DROP policies and how they protect the system from unintended traffic exposure due to missing or misconfigured rules.

Task 2: Building a Decision Tree for Login Anomaly Detection (45 pts)

Objective

In this task, you will build a simple **anomaly-based intrusion detection system** that uses a **decision tree classifier** to analyze login behavior and classify each login attempt as either **normal** or **anomalous**. You will work with a labeled dataset, encode categorical values, train and evaluate the model using accuracy, precision, recall, and F1-score, and reflect on the strengths and limitations of these evaluation metrics.

Step 1: Set Up the Project and Load the Dataset

Objective: Set up your development environment and inspect the login dataset.

1. Download the provided CSV file: [CSV dataset](#)
2. Open **PyCharm**, **Jupyter Notebook**, **VS Code**, or your preferred Python IDE.
3. Create a new Python project or working directory.
4. Add the dataset (Login_Dataset.csv) to your **project root directory**.
5. Create a new Python file named **classifier.py** in the same directory.
6. Run the following command to install the required libraries:

```
pip install pandas scikit-learn
```

Note: this step may be different depending on your IDE.

7. Begin writing your code by importing pandas and loading the dataset:

```
import pandas as pd

df = pd.read_csv("Login_Dataset.csv")
print(df.head())
```

1. *What do each of the columns mean? Which column is the target variable (what we want to predict)?*

Step 2: Encode Categorical Variables

Objective: Convert the categorical columns (Device Type , IP Risk Score , Login Status) into numbers.

```
from sklearn.preprocessing import LabelEncoder

df['Device Type'] = LabelEncoder().fit_transform(df['Device Type'])
df['IP Risk Score'] = LabelEncoder().fit_transform(df['IP Risk Score'])
df['Login Status'] = LabelEncoder().fit_transform(df['Login Status'])
```

2. *Why do we need to encode text into numbers? Can scikit-learn models handle strings directly?*

Step 3: Prepare Features and Target

Objective: Define the input features (X) and the label/output (y).

```
X = df[['Login Hour', 'Device Type', 'Failed Attempts', 'IP Risk Score']]
y = df['Login Status']
```

Step 4: Split into Training and Testing Sets

Objective: Use `train_test_split` to divide the dataset for model evaluation.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

3. *Why do we split data into training and testing sets? What happens if we don't?*

Step 5: Train the Decision Tree Classifier

Objective: Initialize and train the model.

```
from sklearn.tree import DecisionTreeClassifier

clf = DecisionTreeClassifier(max_depth=3, random_state=42)
clf.fit(X_train, y_train)
```

4. What does `max_depth=3` do? Try changing this value later.

Step 6: Make Predictions and Evaluate

Objective: Evaluate the model using accuracy, precision, recall, and F1-score.

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

y_pred = clf.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1 Score:", f1_score(y_test, y_pred))
```

Print the confusion matrix:

```
from sklearn.metrics import confusion_matrix
print(confusion_matrix(y_test, y_pred))
```

Analysis Questions

Write a well-reasoned response addressing the following questions. Support your answers with observations from your code and include screenshots if necessary.

1. What was the **accuracy, Precision, Recall, and F1 Score** of your model? Include a screenshot adding the output of your code.
2. How many total predictions were made?

3. How many **False Positives** and **False Negatives** were made?
4. Can you think of any real-life risks of **false positives** or **false negatives** in this system?
5. What are **precision** and **recall**? How do they differ in what they measure?
6. Why might **accuracy** be misleading when the classes are imbalanced?
7. Why is it important to look at **all three metrics** (accuracy, precision, and recall) rather than relying on just one?
8. What does the **F1-score** tell us? Why is it useful when evaluating classifiers in imbalanced or security-sensitive settings?
9. In your results, was the F1-score closer to precision or recall? What does that imply about your model's trade-off?
10. Suppose an attacker spoofs the most important feature (e.g., sets "IP Risk Score" to the safest value). Would your classifier likely still detect the login as anomalous? Justify using the decision tree logic.
11. If you were to retrain the model using a synthetic attack case with engineered feature values to simulate a stealthy login, what kind of values might you use to bypass detection? Why?
12. Based on this experiment, what are one or two additional features you would recommend collecting in future logs to improve anomaly detection?

Task 3: Secure File Exchange between Alice and Bob – Encryption and Integrity (45 pts)

Objective

In this task, you will implement a **hybrid cryptosystem** that allows **Alice** and **Bob** to exchange a file securely over an untrusted channel.

Your implementation must ensure:

- **Confidentiality** – only the intended recipient can read the file.
- **Integrity** – any tampering is detected.
- **Key protection** – symmetric keys are never exposed in transit.

You will combine **RSA** (for key protection) with **AES-CTR** (for data encryption) and **HMAC-SHA256** (for integrity).

This task builds on your earlier network-defense labs by showing how cryptography enforces the same principles of **least privilege**, **fail-safe defaults**, and **trust boundaries** at the data level.

System Overview

1. **AES-CTR** encrypts the file contents.
2. **HMAC-SHA256** authenticates the ciphertext + IV.
3. **RSA-OAEP (SHA-256)** wraps the AES + HMAC keys and IV for secure delivery.

Implementation Requirements

1. Key Generation (`keygen.py`)

- Generate an **RSA-2048** key pair for **Bob** (the receiver).
- Save to:
 - `bob_public.pem`
 - `bob_private.pem`
- You may optionally generate a key pair for Alice if you later add digital signatures.

2. File Encryption (`encrypt.py`)

- Inputs: `secret.txt` , `bob_public.pem`
- Steps:
 - i. Generate random **AES-256 key**, **HMAC-256 key**, and **16-byte IV**.
 - ii. Encrypt `secret.txt` with **AES-CTR** → ciphertext .
 - iii. Compute **HMAC-SHA256** over (IV || ciphertext) → tag .
 - iv. Concatenate AES_key || HMAC_key || IV → key_bundle .
 - v. Encrypt key_bundle using **RSA-OAEP(SHA-256, MGF1-SHA-256)** and Bob's public key → wrapped_keys.bin .
 - vi. Save outputs:
 - `ciphertext.bin` = IV || ciphertext
 - `tag.bin` = HMAC tag
 - `wrapped_keys.bin` = RSA-OAEP ciphertext

3. File Decryption (`decrypt.py`)

- Inputs: `bob_private.pem` , the three files above.
- Steps:
 - i. Decrypt `wrapped_keys.bin` using **RSA-OAEP** → recover AES key, HMAC key, IV.
 - ii. Verify **HMAC** over (IV || ciphertext) matches `tag.bin` .
 - If mismatch → print "Integrity check failed!" and stop.

iii. If valid, decrypt `ciphertext` using **AES-CTR** with recovered IV → `decrypted.txt` .

Testing and Validation

Perform and document the following tests (with screenshots):

1.  **Successful Round-Trip** – Encrypt + Decrypt → output matches `secret.txt` .
2.  **Tamper Test** – Flip a byte in `ciphertext.bin` → HMAC verification fails.
3.  **Wrong RSA Key Test** – Attempt decryption with another private key → unwrap fails.

Deliverables

- `keygen.py` – RSA key generation
- `encrypt.py` – AES/HMAC encryption + RSA key wrap
- `decrypt.py` – RSA unwrap + HMAC verify + AES decryption
- Output files:
 - `wrapped_keys.bin`
 - `ciphertext.bin`
 - `tag.bin`
 - `decrypted.txt` (for verification)
- `assignment_report.md` – short report with screenshots and answers to questions below.

Analysis Questions

1. Why do we combine **RSA** and **AES** instead of encrypting the file directly with RSA?
2. What would happen if you reused the **IV** in AES-CTR?
3. How does **RSA-OAEP** protect the symmetric keys compared to basic RSA encryption?
4. Which **NIST CSF Function** and **SP 800-53 Rev 5** controls does this system align with?